

Experiment 7

Module Name: Unit Test Frameworks

Experiment Name: Page Object Model (POM) Restructure

Problem statement: Take your working scripts from Tier 1 and restructure them into a Page Object Model design pattern.

Objective: Refactor a standard login script into a Page Object Model (POM) architecture, isolating UI interactions and locators into dedicated classes, keeping the main test script clean and focused purely on assertions.

Tools, software, and concepts used: Python 3, Selenium WebDriver, Object-Oriented Programming (OOP), Page Object Model (POM) Design Pattern.

Source code or implementation steps:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# --- PAGE OBJECT CLASS ---
class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        self.username_input = (By.ID, "user-name")
        self.password_input = (By.ID, "password")
        self.login_btn = (By.ID, "login-button")

    def enter_username(self, username):
        self.driver.find_element(*self.username_input).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password_input).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.login_btn).click()

# --- TEST EXECUTION ---
driver = webdriver.Chrome()
driver.maximize_window()

try:
    driver.get("https://www.saucedemo.com/")

    # Instantiate the Page Object
    login_page = LoginPage(driver)

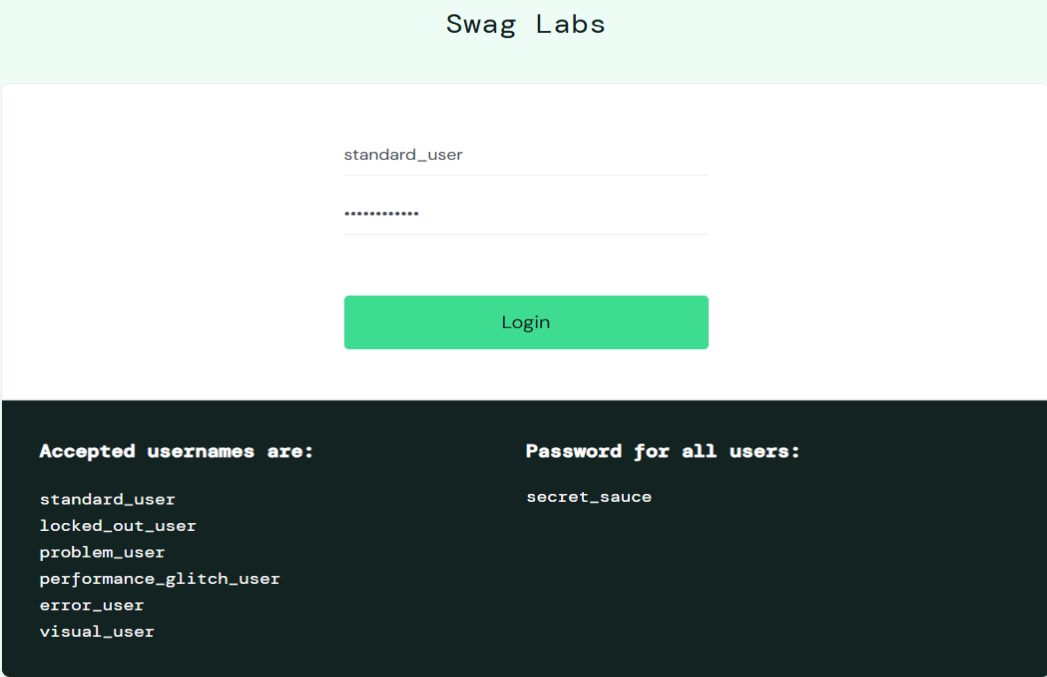
    # Execute actions via the Page Object
    login_page.enter_username("standard_user")
    login_page.enter_password("secret_sauce")
    login_page.click_login()
```

```
time.sleep(2)

# Assertions remain in the test script
assert "inventory.html" in driver.current_url, "Login failed!"
print("PASS: POM Login Successful")

finally:
    driver.quit()
```

Output with clearly labelled screenshots:



The screenshot shows the Swag Labs login page. At the top, there is a light green header with the text "Swag Labs". Below the header is a white login form. The form has two input fields: the first is labeled "standard_user" and contains the text "standard_user"; the second is a password field with masked characters "*****". Below the password field is a green "Login" button. At the bottom of the page, there is a dark green footer with white text. The footer is divided into two columns. The left column is titled "Accepted usernames are:" and lists the following usernames: "standard_user", "locked_out_user", "problem_user", "performance_glitch_user", "error_user", and "visual_user". The right column is titled "Password for all users:" and contains the password "secret_sauce".

Screenshot 1: Successfully Putting Username & Password

Terminal output: PASS: POM Login Successful

Brief Result: The login test executed successfully using methods called from the imported LoginPage class.

Observation: The test_login_pom.py file no longer contains any Selenium By.ID locators, making it strictly a behavior and validation script.

Conclusion: POM is crucial for enterprise-grade automation. It enforces the DRY (Don't Repeat Yourself) principle, ensuring that UI updates only require changing a locator in one single page object file, rather than across hundreds of test scripts.